

نقشه راه و معماری پروژه

توسعه فرانت‌اند با React - تیم ۲ نفره

فهرست مطالب

۱	ساختار پروژه (Project Structure)
۲	تقسیم‌بندی وظایف تیم
۳	فاز ۰: پیش‌نیازها و تصمیمات اولیه
۴	فاز ۱: زیرساخت و راه‌اندازی (هفته ۱-۳)
۵	فاز ۲: Authentication (هفته ۳-۴)
۶	فاز ۳: صفحه اصلی Homepage (هفته ۵-۶)
۷	فاز ۴: سیستم چت خصوصی (هفته ۷-۱۰)
۸	فاز ۵: Account و Profile (هفته ۱۱-۱۲)
۹	فاز ۶: Contacts و Notifications (هفته ۱۳-۱۴)
۱۰	فاز ۷: سیستم تماس (هفته ۱۵-۱۶)
۱۱	فاز ۸: بهینه‌سازی Performance (هفته ۱۷-۱۸)

۱۲	فاز ۹: تست و Debugging (هفته ۱۹)
۱۳	فاز ۱۰: آماده‌سازی Production (هفته ۲۰)
۱۴	جلسات و Git Workflow
۱۵	ریسک‌ها و راه‌حل‌ها
۱۶	تعریف Done برای هر تسک

۱. ساختار پروژه

معماری Feature-Based برای مقیاس‌پذیری و توسعه تیمی

ساختار کامل پوشه‌ها

```

my-project/
├── public/
│   └── favicon.ico
├── src/
│   ├── assets/                                # فایل‌های استاتیک (تصاویر، فونت، ویدیو)
│   │   ├── images/
│   │   ├── fonts/
│   │   └── videos/
│   ├── components/                            # کامپوننت‌های مشترک و قابل استفاده مجدد
│   │   ├── ui/                                # UI کامپوننت‌های پایه
│   │   │   ├── Button.jsx
│   │   │   ├── Input.jsx
│   │   │   ├── Modal.jsx
│   │   │   ├── Card.jsx
│   │   │   ├── Avatar.jsx
│   │   │   ├── Badge.jsx
│   │   │   ├── Spinner.jsx
│   │   │   └── Toast.jsx
│   │   ├── layout/                            # کامپوننت‌های ساختاری
│   │   │   ├── Header.jsx
│   │   │   ├── Footer.jsx
│   │   │   ├── Sidebar.jsx
│   │   │   ├── NavMenu.jsx
│   │   │   └── MainLayout.jsx
│   │   └── search/
│   │       └── SearchBar.jsx
│   ├── features/                              # (Feature-Based معماری) فیچرهای اصلی
│   │   ├── auth/                             # احراز هویت
│   │   │   ├── components/
│   │   │   │   ├── LoginForm.jsx
│   │   │   │   └── RegisterForm.jsx
│   │   │   ├── pages/
│   │   │   │   ├── LoginPage.jsx
│   │   │   │   └── RegisterPage.jsx
│   │   │   └── hooks/
│   │   │       └── useAuth.js
│   │   ├── home/                             # صفحه اصلی
│   │   │   ├── components/
│   │   │   │   ├── Category.jsx
│   │   │   │   ├── Profiles.jsx
│   │   │   │   ├── ProfileSelection.jsx
│   │   │   │   ├── BoxOne.jsx
│   │   │   │   ├── BoxTwo.jsx
│   │   │   │   └── BoxThree.jsx

```

```

└─ pages/
    └─ HomePage.jsx

└─ chat/ # چت خصوصی
    └─ components/
        └─ HeaderPv.jsx
        └─ PrivateChat.jsx
        └─ TypewriteBox.jsx
        └─ ChatBubble.jsx
        └─ ChatList.jsx
    └─ pages/
        └─ ChatRoom.jsx
    └─ hooks/
        └─ useChat.js

└─ account/ # حساب کاربری
    └─ components/
    └─ pages/
        └─ AccountPage.jsx

└─ calls/ # تماس
    └─ components/
    └─ pages/
        └─ CallPage.jsx

└─ contacts/ # مخاطبین
    └─ pages/
        └─ ContactsPage.jsx

└─ notifications/ # اعلان‌ها
    └─ components/
    └─ pages/
        └─ NotificationsPage.jsx

└─ hooks/ # هوک‌های سفارشی عمومی
    └─ useLocalStorage.js
    └─ useFetch.js
    └─ useDebounce.js
    └─ useOnlineStatus.js

└─ services/ # سرویس‌های API
    └─ api.js # axios پیکربندی
    └─ authService.js
    └─ chatService.js
    └─ notificationService.js

└─ context/ # Context API ها (State Management)
    └─ AuthContext.jsx
    └─ ChatContext.jsx
    └─ ThemeContext.jsx

└─ routes/ # مسیرهای برنامه
    └─ AppRouter.jsx
    └─ ProtectedRoute.jsx

```

```
|   ├── utils/                                # توابع کمکی
|   |   ├── formatDate.js
|   |   ├── validators.js
|   |   └── jalali.js
|   |
|   ├── constants/                          # مقادیر ثابت
|   |   ├── routes.js
|   |   ├── config.js
|   |   └── actionTypes.js
|   |
|   ├── styles/                             # استایل‌های سراسری
|   |   └── globals.css
|   |
|   ├── App.jsx                             # کامپوننت اصلی
|   └── main.jsx                             # نقطه ورود برنامه
|
| ├── .gitignore
| ├── .env.development
| ├── .env.production
| ├── eslint.config.js
| ├── index.html
| ├── package.json
| ├── postcss.config.js
| ├── README.md
| ├── tailwind.config.js
| └── vite.config.js
```

۲. تقسیم‌بندی وظایف تیم

نقش‌ها و مسئولیت‌های دو توسعه‌دهنده

Person A - Tech Lead / Senior Developer 🧑🏻💻

- راه‌اندازی پروژه و پیکربندی ابزارها
- معماری کلی پروژه و ساختار پوشه‌ها
- پیاده‌سازی Context API و State Management
- نوشتن سرویس‌های API و لایه ارتباط با سرور
- راه‌اندازی WebSocket برای چت
- پیاده‌سازی WebRTC برای تماس
- مدیریت Performance و بهینه‌سازی
- تنظیم CI/CD و آماده‌سازی Production
- Code Review نهایی

Person B - UI/UX Developer 🧑🏻💻

- طراحی و پیاده‌سازی UI Kit کامل
- ساخت کامپوننت‌های پایه (Button, Input, Modal, ...)
- طراحی صفحات Login و Register
- پیاده‌سازی صفحه اصلی با تمام کامپوننت‌ها
- طراحی رابط کاربری چت (Chat UI)
- طراحی صفحات Profile و Account
- انیمیشن‌ها و میکرو-اینتراکشن‌ها
- تست Responsive و Cross-browser
- پالیش نهایی UI/UX

۳. فاز ۰: پیش‌نیازها و تصمیمات اولیه

قبل از شروع کدنویسی - جلسه مشترک ۲ ساعته

تصمیمات فنی

State Management: Context API + useReducer

Form Management: React Hook Form + Zod

Interceptor یا HTTP Client: Axios

Styling: Tailwind CSS + CSS Modules

WebSocket: Socket.io-client

Agora SDK یا WebRTC: Simple-peer

Linting: ESLint + Prettier

Git Hooks: Husky + lint-staged

Testing: Vitest + React Testing Library

命名 Convention: PascalCase کامپوننت‌ها، camelCase توابع

Git Workflow: Git Flow (main ← develop ← feature/*)

Commit Convention: Conventional Commits

۴. فاز ۱: زیرساخت و راه‌اندازی

هفته ۱ و ۲ | ایجاد foundation پروژه

Person A 🧑

- ایجاد پروژه با Vite
- نصب تمام dependency های اصلی
- تنظیم ESLint + Prettier
- تنظیم Tailwind CSS
- ایجاد فایل‌های محیطی (env.)
- تنظیم aliases در vite.config
- ایجاد ساختار پوشه‌ها
- ایجاد فایل‌های main (App.jsx, main.jsx)
- راه‌اندازی React Router
- ایجاد ProtectedRoute
- تنظیم axios با interceptorها
- ایجاد api service اولیه
- راه‌اندازی Context های اصلی (Auth, Theme)
- تنظیم Git hooks (Husky)
- نوشتن README و مستندات
- ایجاد GitHub PR template

Person B 🧑

- آماده‌سازی Design System در Figma
- استخراج Token های طراحی
- تنظیم Tailwind config (colors, fonts)

- ایجاد فایل‌های CSS گلوبال
- ساخت Button (5 variant - 4 size)
- ساخت Input (با label, error, icon)
- ساخت Modal (3 size یا overlay)
- ساخت Card component
- ساخت Avatar component
- ساخت Badge component
- ساخت Spinner/Loading
- ساخت Skeleton Loader
- ساخت Toast/Notification
- ساخت Dropdown
- ساخت Tooltip

✓ نقاط چک مشترک

پروژه بدون error اجرا میشه

ESLint و Prettier کار میکنن

Tailwind درست تنظیم شده

Router کار میکنه

UI Kit کامل و آماده است

۵. فاز ۲: Authentication

هفته ۳ و ۴ | سیستم احراز هویت

Person A - منطق و سرویس‌ها 🧑

- تکمیل AuthContext (login, register, logout, refreshToken)
- ذخیره token در localStorage
- مدیریت state کاربر
- ساخت authService (API calls)
- پیاده‌سازی interceptor برای refresh token
- مدیریت 401 و redirect خودکار
- پیاده‌سازی validation logic با Zod
- ساخت schemaهای login و register
- مدیریت خطاهای سرور
- نمایش toast برای خطاها
- مدیریت loading states
- Unit test برای auth logic

Person B - UI و Form ها 🧑

- طراحی و پیاده‌سازی LoginPage
- Form validation و نمایش خطاها
- Responsive design برای mobile
- طراحی و پیاده‌سازی RegisterPage
- Password strength meter
- Confirm password logic
- طراحی ForgotPassword (اختیاری)

Email verification UI ○

انیمیشن‌های transition بین صفحات auth ○

skeleton و Loading states ○

empty states و Error states ○

نقاط چک مشترک ✓

کاربر میتونه لاگین کنه و token ذخیره بشه

کاربر میتونه ثبت نام کنه

بعد از refresh صفحه، user state حفظ میشه

خطاهای اعتبارسنجی درست نمایش داده میشن

Protected routes کار میکنن

Logout درست کار میکنه

۶. فاز ۳: صفحه اصلی (Homepage)

هفته ۵ و ۶ | صفحه اصلی برنامه

Person A - منطق و دیتا 🧑

- ساخت custom hook برای fetch categories
- پیاده‌سازی search logic
- مدیریت loading/error/empty states
- ساخت useDebounce برای search
- پیاده‌سازی infinite scroll یا pagination
- اتصال به API واقعی (categories, profiles)
- React Query با Caching strategy
- بهینه‌سازی performance با React.memo
- استفاده از useMemo/useCallback
- Person B با Code review

Person B - UI 🧑

- ساخت Searchbar با autocomplete
- ساخت Category cards
- ساخت Profile cards
- ساخت ProfileSelection component
- ساخت BoxOne, BoxTwo, BoxThree
- چیدمان responsive با grid
- انیمیشن‌های hover و transition
- Skeleton loading برای کارت‌ها
- بهینه‌سازی mobile view

○ تست cross-browser

✓ نقاط چک مشترک

Search کار میکنه (با debounce)

Categories از API لود میشن

Profiles نمایش داده میشن

Responsive در mobile و tablet

Loading states داریم

Error handling داریم

۷. فاز ۴: سیستم چت خصوصی

هفته ۷ تا ۱۰ | پیچیده‌ترین بخش پروژه

هفته ۷-۸: زیرساخت چت

Person A

- ☐ راه‌اندازی WebSocket connection
- ☐ ساخت ChatContext
- ☐ مدیریت state پیام‌ها (send, receive, delete)
- ☐ Load history messages
- ☐ Mark as read logic
- ☐ Reconnection strategy
- ☐ Offline message queue

Person B

- ☐ طراحی UI لیست چت‌ها (ChatList)
- ☐ Chat preview component
- ☐ Online/offline status indicator
- ☐ Unread message badge
- ☐ Search in conversations
- ☐ Sort و filter conversations

هفته ۹-۱۰: صفحه چت

Person A

- Real-time message sending/receiving ☐
- Message encryption (در صورت نیاز) ☐
- File/Image upload logic ☐
- Typing indicator logic ☐
- Read receipts ☐
- Message pagination (load more) ☐

Person B

- Chat bubble designs ☐
- Message input با emoji picker ☐
- File attachment UI ☐
- Typing indicator animation ☐
- Scroll to bottom logic ☐
- Message actions (copy, delete, reply) ☐

۸. فاز ۵ و ۶: Profile و Notifications

هفته ۱۱ تا ۱۴ | پروفایل، اعلان‌ها و مخاطبین

هفته ۱۱-۱۲: Account و Profile

Person A

- Profile service (get/update profile) ☐
- آپلود عکس پروفایل ☐
- Change password logic ☐
- Privacy settings logic ☐
- Activity log ☐
- Session management ☐

Person B

- Profile page layout ☐
- Edit profile form ☐
- Profile picture uploader ☐
- Password change form ☐
- Settings tabs design ☐
- Privacy controls UI ☐

هفته ۱۳-۱۴: Contacts و Notifications

Person A

Notification service (get, mark read, delete) ☐

Real-time notifications via WebSocket ☐

Contacts service (add, remove, block) ☐

Contact sync logic ☐

Person B

Notification center design ☐

Notification cards (انواع مختلف) ☐

Contacts list design ☐

Add contact modal ☐

Block/Unblock UI ☐

۹. فاز ۷: سیستم تماس (Call)

هفته ۱۵ و ۱۶ | پیچیده‌ترین بخش با WebRTC

Person A - WebRTC Logic 🧑

- WebRTC peer connection setup ○
- Signaling server integration ○
- ICE candidate handling ○
- Call state machine (ringing, connected, ended) ○
- Mute/Unmute audio logic ○
- Video toggle logic ○
- Screen sharing (در صورت نیاز) ○

Person B - Call UI 🧑

- Call UI (incoming/outgoing) ○
- Call controls (mute, speaker, end) ○
- Call timer ○
- Ringing animation ○
- Full-screen vs picture-in-picture ○
- Video grid برای group calls ○

۱۰. فاز ۸: بهینه‌سازی و Performance

هفته ۱۷ و ۱۸ | Audit و بهبود

هفته ۱۷: Performance Audit

- ☐ Lighthouse audit و رفع مشکلات
- ☐ Code splitting (lazy loading routes)
- ☐ Image optimization (WebP, lazy loading)
- ☐ Bundle size optimization
- ☐ Tree shaking بررسی
- ☐ Memoization بررسی

هفته ۱۸: UX Improvements

- ☐ PWA setup (service worker, manifest)
- ☐ Offline mode
- ☐ Error boundaries
- ☐ page 404
- ☐ Empty states (همه جا)
- ☐ Loading skeletons (همه جا)
- ☐ انیمیشن‌های Micro-interaction

۱۱. فاز ۹: تست و Debugging

هفته ۱۹ | تضمین کیفیت

Person A 🧑

- Unit tests (hooks, utils, services) ☐
- Integration tests (auth flow, chat flow) ☐
- E2E tests (critical paths) ☐
- Bug fixing از لیست issues ☐

Person B 🧑

- Visual regression testing ☐
- Cross-browser testing ☐
- Mobile responsive testing ☐
- Accessibility audit (a11y) ☐
- Keyboard navigation ☐

۱۲. فاز ۱۰: آماده‌سازی Production

هفته ۲۰ | Deployment نهایی

Person A - DevOps 🧑💻

- Environment variables final ○
- Build optimization ○
- CI/CD pipeline final ○
- Error tracking setup (Sentry) ○
- Analytics setup ○
- SSL/HTTPS check ○
- Monitoring setup ○
- Backup strategy ○

Person B - Final Polish 🧑💻

- Final UI polish ○
- meta tags و Favicon ○
- Social media preview (OG tags) ○
- sitemap.xml ○
- robots.txt ○
- page custom 404 ○
- مستندات نهایی ○
- Release notes ○

۱۳. جلسات منظم و Git Workflow

جلسات تیمی

Daily Standup (۱۵ دقیقه - هر روز)

- ☐ دیروز چی کار کردی؟
- ☐ امروز چی کار میکنی؟
- ☐ مشکلی هست؟

Weekly Review (۱ ساعت - آخر هفته)

- ☐ Review کارهای انجام شده
- ☐ Demo به Product Owner
- ☐ برنامه‌ریزی هفته بعد
- ☐ بررسی Technical Debt

Git Workflow

```
main (production)
└─ develop (integration)
    ├── feature/auth
    ├── feature/homepage
    ├── feature/chat
    ├── feature/profile
    ├── feature/notifications
    └─ feature/call
```

قوانین:

- خودش branch تو feature هر
- code review یا develop به PR
- Merge فقط برای main به release
- اجباری Conventional Commits
- merge قبل از approve حداقل ۱

Conventional Commits

```
feat:    ویژگی جدید
fix:     رفع باگ
docs:    تغییرات مستندات
style:   فرمت کد (بدون تغییر در منطق)
refactor: بازنویسی کد
test:    اضافه کردن تست
chore:   و ابزارها build تغییرات
perf:    بهبود performance
```

مثال:

```
feat(auth): add login page
fix(chat): resolve message duplication
```

۱۴. ریسک‌ها و راه‌حل‌ها

ریسک	احتمال	شدت تأثیر	راه‌حل
API آماده نیست	بالا	متوسط	استفاده از Mock Service Worker (MSW)
WebSocket پیچیدگی بالا	متوسط	بالا	استفاده از Socket.io library
WebRTC پیچیده بشه	بالا	بالا	استفاده از سرویس‌های آماده (Agora, Twilio)
Performance مشکل	متوسط	متوسط	مانیتورینگ مداوم از همون اول
Conflict های Git	بالا	پایین	Pull request کوچک و Frequent merge
تأخیر در طراحی Figma	پایین	متوسط	شروع با wireframe و بهبود تدریجی

۵. تعریف Done برای هر تسک

✓ چک‌لیست تکمیل هر تسک

کد نوشته شده

بدون خطا ESLint

Responsive طراحی (mobile, tablet, desktop)

Loading state پیاده‌سازی شده

Error state پیاده‌سازی شده

Empty state پیاده‌سازی شده

Test نوشته شده (حداقل برای منطق اصلی)

Code Review شده

تایید از نفر دیگر گرفته شده

در develop branch مرج شده

Conflict با develop ندارد

مستندات آپدیت شده (در صورت نیاز)